MATH 128 - HW13 - Jie Wen

Problem 1: Romberg Integration

2

(a) $\int_{-1}^{1}(\cos x)^2 dx$

Let $f(x)=\cos^2 x$ on $[-1,1]$.
The interval length and step sizes are:

- $a=-1, b=1, b-a=2$
- $h_1=2, h_2=1, h_3=\frac{1}{2}$.

The function values (rounded) are:

$$\begin{gathered} f(-1)=f(1)=\cos^2(1)\approx 0.29193, \\ f(0)=1, \qquad f(\pm 0.5)=\cos^2(0.5)\approx 0.77015. \end{gathered}$$

$$R_{1,1}=\frac{h_1}{2}[f(-1)+f(1)]=\frac{2}{2}[f(-1)+f(1)]=2\cos^2(1)\approx 0.58385.$$

$$R_{2,1}=\frac{h_2}{2}[f(-1)+2f(0)+f(1)]=\frac{1}{2}[2\cos^2(1)+2]\approx 1.29193.$$

$$R_{3,1}=\frac{h_3}{2}\big[f(-1)+2f(-0.5)+2f(0)+2f(0.5)+f(1)\big]\approx 1.41611.$$

$$\begin{aligned} R_{2,2}&=R_{2,1}+\frac{R_{2,1}-R_{1,1}}{3}\approx 1.29193+\frac{1.29193-0.58385}{3}\approx 1.52795, \\ \therefore R_{3,2}&=R_{3,1}+\frac{R_{3,1}-R_{2,1}}{3}\approx 1.41611+\frac{1.41611-1.29193}{3}\approx 1.45751, \\ R_{3,3}&=R_{3,2}+\frac{R_{3,2}-R_{2,2}}{15}\approx 1.45751+\frac{1.45751-1.52795}{15}\approx \boxed{1.45281}. \end{aligned}$$

(b) $\int_{-0.75}^{0.75} x\ln(x+1)dx$

Let $f(x)=x\ln(x+1)$ on $[-0.75,0.75]$.
The parameters are:

- $a=-0.75, b=0.75, b-a=1.5$
- $h_1=1.5, h_2=0.75, h_3=0.375$.

The function values (rounded) are:

$$\begin{gathered} f(-0.75)=-0.75\ln(0.25)\approx 1.03972, \\ f(0)=0, \\ f(0.75)=0.75\ln(1.75)\approx 0.41971, \\ f(-0.375)=-0.375\ln(0.625)\approx 0.17625, \\ f(0.375)=0.375\ln(1.375)\approx 0.11942. \end{gathered}$$

$$R_{1,1}=\frac{h_1}{2}[f(-0.75)+f(0.75)]=0.75[1.03972+0.41971]\approx 1.09457.$$

$$R_{2,1}=\frac{h_2}{2}[f(-0.75)+2f(0)+f(0.75)]=0.375[1.03972+0+0.41971]\approx 0.54729.$$

$$R_{3,1}=\frac{h_3}{2}\big[f(-0.75)+2f(-0.375)+2f(0)+2f(0.375)+f(0.75)\big]\approx 0.38452.$$

$$\begin{aligned} R_{2,2}&=R_{2,1}+\frac{R_{2,1}-R_{1,1}}{3}\approx 0.54729+\frac{0.54729-1.09457}{3}\approx 0.36486, \\ \therefore R_{3,2}&=R_{3,1}+\frac{R_{3,1}-R_{2,1}}{3}\approx 0.38452+\frac{0.38452-0.54729}{3}\approx 0.33026, \\ R_{3,3}&=R_{3,2}+\frac{R_{3,2}-R_{2,2}}{15}\approx 0.33026+\frac{0.33026-0.36486}{15}\approx \boxed{0.32796}. \end{aligned}$$

12

Let

$$a=f(0), \quad b=f(1), \quad c=f\left(\tfrac{1}{2}\right).$$

For one subinterval ($h=1$), the trapezoidal rule gives

$$R_{11}=\frac{1}{2}[f(0)+f(1)]=\frac{1}{2}(a+b)=4 \Longrightarrow a+b=8. \tag{1}$$

For two subintervals ($h = \frac{1}{2}$), the composite trapezoidal rule gives

$$R_{21} = \frac{h}{2}\left[f(0) + 2f\left(\tfrac{1}{2}\right) + f(1)\right] = \frac{1}{4}(a + 2c + b). \tag{2}$$

Romberg extrapolation to get R_{22} is

$$R_{22} = \frac{4R_{21} - R_{11}}{3}.$$

Given $R_{22} = 5$ and $R_{11} = 4$,

$$5 = \frac{4R_{21} - 4}{3} \Longrightarrow R_{21} = \frac{19}{4}. \tag{3}$$

From (2) and (3),

$$\frac{1}{4}(a + 2c + b) = \frac{19}{4} \Longrightarrow a + 2c + b = 19. \tag{4}$$

Subtract (1) from (4):

$$c = f\left(\tfrac{1}{2}\right) = \frac{11}{2}.$$

17

Let

$$h_k = \frac{b-a}{2^{k-1}}, \qquad n_k = 2^{k-1},$$

and $x_j = a + jh_k$, for $j = 0, 1, \ldots, n_k$.

The composite trapezoidal rule with step h_k (i.e., using n_k subintervals) is

$$R_{k,1} = \frac{h_k}{2}\Big[f(x_0) + 2\sum_{j=1}^{n_k-1} f(x_j) + f(x_{n_k})\Big].$$

The previous trapezoidal approximation $R_{k-1,1}$ uses step size $h_{k-1} = 2h_k$, with $n_{k-1} = 2^{k-2} = n_k/2$ subintervals:

$$R_{k-1,1} = \frac{h_{k-1}}{2}\Big[f(x_0) + 2\sum_{j=1}^{n_{k-1}-1} f(x_{2j}) + f(x_{n_k})\Big] = h_k\Big[f(x_0) + 2\sum_{j=1}^{n_k/2-1} f(x_{2j}) + f(x_{n_k})\Big].$$

Here, the nodes of $R_{k-1,1}$ are expressed in terms of the finer grid x_j with step h_k; they correspond to the even-indexed nodes x_{2j}.

For $R_{k,1}$, the interior sum is split into even and odd indices:

- Even interior nodes:

$$S_{\text{even}} = \sum_{j=1}^{n_k/2-1} f(x_{2j}).$$

- Odd interior nodes:

$$S_{\text{odd}} = \sum_{j=1}^{n_k/2} f(x_{2j-1}).$$

Then

$$\sum_{j=1}^{n_k-1} f(x_j) = S_{\text{even}} + S_{\text{odd}}.$$

Substituting this into the expression for $R_{k,1}$:

$$R_{k,1} = \frac{h_k}{2}\Big[f(x_0) + 2(S_{\text{even}} + S_{\text{odd}}) + f(x_{n_k})\Big].$$

The expression for $R_{k-1,1}$ is:

$$R_{k-1,1} = h_k\Big[f(x_0) + 2S_{\text{even}} + f(x_{n_k})\Big].$$

Romberg's extrapolation for the second column is given by

$$R_{k,2} = R_{k,1} + \frac{1}{3}(R_{k,1} - R_{k-1,1}) = \frac{4R_{k,1} - R_{k-1,1}}{3}.$$

Next, subtract $R_{k-1,1}$:

$$\begin{aligned}4R_{k,1} - R_{k-1,1} &= 2h_k\Big[f(x_0) + 2S_{\text{even}} + 2S_{\text{odd}} + f(x_{n_k})\Big] - h_k\Big[f(x_0) + 2S_{\text{even}} + f(x_{n_k})\Big] \\ &= h_k\Big[(2-1)f(x_0) + (4-2)S_{\text{even}} + 4S_{\text{odd}} + (2-1)f(x_{n_k})\Big] \\ &= h_k\Big[f(x_0) + 2S_{\text{even}} + 4S_{\text{odd}} + f(x_{n_k})\Big].\end{aligned}$$

Therefore,

$$R_{k,2} = \frac{4R_{k,1} - R_{k-1,1}}{3} = \frac{h_k}{3}\Big[f(x_0) + 2S_{\text{even}} + 4S_{\text{odd}} + f(x_{n_k})\Big].$$

The Composite Simpson's rule with step size $h = h_k$ and even n_k is:

$$\int_a^b f(x)dx \approx \frac{h_k}{3}\left[f(x_0) + 2\sum_{j=1}^{n_k/2-1} f(x_{2j}) + 4\sum_{j=1}^{n_k/2} f(x_{2j-1}) + f(x_{n_k})\right].$$

This expression is identical to

$$\frac{h_k}{3}\Big[f(x_0) + 2S_{\text{even}} + 4S_{\text{odd}} + f(x_{n_k})\Big],$$

which was shown to be equal to $R_{k,2}$.

Hence, **the Romberg approximation $R_{k,2}$ is identical to the Composite Simpson's rule with step size h_k.**

18

The sum on the left-hand side is split into terms with odd and even indices. Since 2^{k-1} is even, the sum can be written as:

$$\sum_{i=1}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right) = \sum_{\substack{i=1 \\ i\text{ odd}}}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right) + \sum_{\substack{i=1 \\ i\text{ even}}}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right).$$

1. **Odd indices.**
   Let $i = 2j-1$, where $j = 1, 2, \ldots, 2^{k-2}$. Then

   $$\frac{i}{2}h_{k-1} = \frac{2j-1}{2}h_{k-1} = \left(j - \frac{1}{2}\right)h_{k-1},$$

   so

   $$\sum_{\substack{i=1 \\ i\text{ odd}}}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right) = \sum_{j=1}^{2^{k-2}} f\left(a + \left(j - \frac{1}{2}\right)h_{k-1}\right).$$

2. **Even indices.**
   Let $i = 2j$, where $j = 1, 2, \ldots, 2^{k-2}$. Then

   $$\frac{i}{2}h_{k-1} = \frac{2j}{2}h_{k-1} = jh_{k-1},$$

   so

   $$\sum_{\substack{i=1 \\ i\text{ even}}}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right) = \sum_{j=1}^{2^{k-2}} f(a + jh_{k-1}).$$

Combining these two parts yields

$$\sum_{i=1}^{2^{k-1}} f\left(a + \frac{i}{2}h_{k-1}\right) = \sum_{i=1}^{2^{k-2}} f\left(a + \left(i - \frac{1}{2}\right)h_{k-1}\right) + \sum_{i=1}^{2^{k-2}} f(a + ih_{k-1}),$$

which is the desired identity.

19

$R_{k,1}$ is the Composite Trapezoidal rule approximation with step size $h_k = \dfrac{b-a}{2^{k-1}}$, so $h_k = \dfrac{h_{k-1}}{2}$.

For step size h_k, the Composite Trapezoidal rule on $[a, b]$ with 2^{k-1} subintervals is

$$R_{k,1} = \frac{h_k}{2}\left[f(a) + 2\sum_{j=1}^{2^{k-1}-1} f\big(a + jh_k\big) + f(b)\right].$$

Since $h_k = \dfrac{h_{k-1}}{2}$, this becomes

$$R_{k,1} = \frac{h_{k-1}}{4}\left[f(a) + 2\sum_{j=1}^{2^{k-1}-1} f\big(a + j\tfrac{h_{k-1}}{2}\big) + f(b)\right].$$

Note that $b = a + 2^{k-1}h_k = a + 2^{k-2}h_{k-1}$.

The sum extends only up to $2^{k-1} - 1$. Adding and subtracting the missing endpoint term yields:

$$\sum_{j=1}^{2^{k-1}-1} f\left(a + j\frac{h_{k-1}}{2}\right) = \sum_{j=1}^{2^{k-1}} f\left(a + j\frac{h_{k-1}}{2}\right) - f(b).$$

Substituting into $R_{k,1}$:

$$\begin{aligned} R_{k,1} &= \frac{h_{k-1}}{4}\Big[f(a) + 2\Big(\sum_{j=1}^{2^{k-1}} f\left(a + j\frac{h_{k-1}}{2}\right) - f(b)\Big) + f(b)\Big] \\ &= \frac{h_{k-1}}{4}\Big[f(a) - f(b) + 2\sum_{j=1}^{2^{k-1}} f\left(a + j\frac{h_{k-1}}{2}\right)\Big]. \end{aligned}$$

For any k,

$$\sum_{j=1}^{2^{k-1}} f\left(a + j\frac{h_{k-1}}{2}\right) = \sum_{i=1}^{2^{k-2}} f\left(a + \left(i - \tfrac{1}{2}\right)h_{k-1}\right) + \sum_{i=1}^{2^{k-2}} f(a + ih_{k-1}).$$

Let

$$S_{\text{mid}} = \sum_{i=1}^{2^{k-2}} f\left(a + \left(i - \tfrac{1}{2}\right)h_{k-1}\right), \quad S_{\text{old}} = \sum_{i=1}^{2^{k-2}} f(a + ih_{k-1}).$$

Then

$$\sum_{j=1}^{2^{k-1}} f\left(a + j\frac{h_{k-1}}{2}\right) = S_{\text{mid}} + S_{\text{old}}.$$

So

$$R_{k,1} = \frac{h_{k-1}}{4}\Big[f(a) - f(b) + 2S_{\text{mid}} + 2S_{\text{old}}\Big].$$

Splitting S_{old} into the old internal points plus the endpoint b:

$$S_{\text{old}} = \sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) + f(b).$$

Thus

$$\begin{aligned} f(a) - f(b) + 2S_{\text{mid}} + 2S_{\text{old}} &= f(a) - f(b) + 2S_{\text{mid}} + 2\sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) + 2f(b) \\ &= f(a) + f(b) + 2\sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) + 2S_{\text{mid}}. \end{aligned}$$

Therefore

$$R_{k,1} = \frac{h_{k-1}}{4}\Big[f(a) + f(b) + 2\sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) + 2S_{\text{mid}}\Big].$$

The Composite Trapezoidal rule with step h_{k-1} (and 2^{k-2} subintervals) is

$$R_{k-1,1} = \frac{h_{k-1}}{2}\Big[f(a) + 2\sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) + f(b)\Big].$$

So

$$f(a) + f(b) + 2\sum_{i=1}^{2^{k-2}-1} f(a + ih_{k-1}) = \frac{2}{h_{k-1}}R_{k-1,1}.$$

Substituting this into the expression for $R_{k,1}$:

$$\begin{aligned} R_{k,1} &= \frac{h_{k-1}}{4}\left[\frac{2}{h_{k-1}}R_{k-1,1} + 2S_{\text{mid}}\right] \\ &= \frac{1}{2}R_{k-1,1} + \frac{h_{k-1}}{2}S_{\text{mid}} \\ &= \frac{1}{2}\left[R_{k-1,1} + h_{k-1}\sum_{i=1}^{2^{k-2}} f\left(a + \left(i - \tfrac{1}{2}\right)h_{k-1}\right)\right]. \end{aligned}$$

This is exactly the desired recurrence formula:

$$R_{k,1} = \frac{1}{2}\left[R_{k-1,1} + h_{k-1}\sum_{i=1}^{2^{k-2}} f\left(a + \left(i - \frac{1}{2}\right)h_{k-1}\right)\right].$$

Thus, Eq. (4.34) is verified.

Problem 2: ODEs

1

(a) $y' = y\cos t,\ 0 \le t \le 1,\ y(0) = 1$.

Let

$$f(t, y) = y\cos t.$$

- **Continuity in (t, y):** The function $\cos t$ is continuous in t, and the map $y \mapsto y\cos t$ is linear in y. Therefore, $f(t, y)$ is continuous on all of $\mathbb{R}^2$, and in particular on the domain $D = \{(t, y) \mid 0 \le t \le 1,\ -\infty < y < \infty\}$.
- **Lipschitz condition in y:**
  The partial derivative with respect to y is:

  $$\frac{\partial f}{\partial y}(t, y) = \cos t.$$

  On the interval $0 \le t \le 1$, it holds that $|\cos t| \le 1$. Thus,

  $$\left|\frac{\partial f}{\partial y}(t, y)\right| \le 1 \quad \text{for all } (t, y) \in D.$$

  By Theorem 5.3, this implies f is Lipschitz in y with Lipschitz constant $L = 1$ on D.

Consequently, the hypotheses of Theorem 5.4 are satisfied on $[0, 1]$, implying **there exists a unique solution** on $0 \le t \le 1$.

The equation is separable:

$$y' = y\cos t \quad \Rightarrow \quad \frac{dy}{dt} = y\cos t.$$

Assuming $y \neq 0$, separating variables yields:

$$\frac{1}{y}dy = \cos t dt.$$

Integrating both sides:

$$\int \frac{1}{y}dy = \int \cos t dt \quad \Rightarrow \quad \ln|y| = \sin t + C.$$

Exponentiating gives:

$$|y| = e^{\sin t + C} = C_1 e^{\sin t},$$

where $C_1 = e^C > 0$. Allowing for the sign, the general solution is

$$y(t) = Ce^{\sin t}.$$

Using the initial condition $y(0) = 1$:

$$1 = y(0) = Ce^{\sin 0} = Ce^0 = C \quad \Rightarrow \quad C = 1.$$

The unique solution is

$$y(t) = e^{\sin t}, \quad 0 \le t \le 1.$$

(d) $y' = \dfrac{4t^3 y}{1 + t^4},\ 0 \le t \le 1,\ y(0) = 1.$

Let

$$f(t, y) = \frac{4t^3}{1 + t^4}y.$$

- **Continuity:** For $0 \le t \le 1$, the denominator $1 + t^4 \neq 0$, so the term $\frac{4t^3}{1+t^4}$ is continuous in t. Multiplying by y ensures $f(t, y)$ is continuous in both variables on the domain $D = \{(t, y) \mid 0 \le t \le 1,\ -\infty < y < \infty\}$.
- **Lipschitz condition in y:**

  $$\frac{\partial f}{\partial y}(t, y) = \frac{4t^3}{1 + t^4}.$$

On the interval $0 \le t \le 1$, this function is continuous and bounded. Specifically,

$$\left|\frac{4t^3}{1+t^4}\right| \le 4 \quad \text{for } 0 \le t \le 1,$$

so a Lipschitz constant $L = 4$ can be used. Thus condition (5.1) holds, and by Theorem 5.3, f is Lipschitz in y on D.

Therefore, by Theorem 5.4, **there exists a unique solution** on $0 \le t \le 1$.

The equation is separable:

$$y' = \frac{4t^3}{1+t^4}y \quad \Rightarrow \quad \frac{1}{y}dy = \frac{4t^3}{1+t^4}dt.$$

Integrating both sides:

$$\int \frac{1}{y}dy = \int \frac{4t^3}{1+t^4}dt.$$

Let $u = 1 + t^4$, so $du = 4t^3 dt$. Then

$$\int \frac{4t^3}{1+t^4}dt = \int \frac{1}{u}du = \ln|u| + C = \ln(1+t^4) + C,$$

since $1 + t^4 > 0$.
So

$$\ln|y| = \ln(1+t^4) + C.$$

Exponentiating yields:

$$|y| = e^C(1+t^4) = C_1(1+t^4).$$

Thus

$$y(t) = C(1+t^4).$$

Applying the initial condition $y(0) = 1$:

$$1 = y(0) = C(1+0^4) = C \quad \Rightarrow \quad C = 1.$$

The unique solution is

$$y(t) = 1 + t^4, \quad 0 \le t \le 1.$$

4

The domain is defined as $D = \{(t, y) \mid 0 \le t \le 1, \ -\infty < y < \infty\}$.

(b) $f(t, y) = \dfrac{1+y}{1+t}$

(i) Lipschitz on D?

The partial derivative with respect to y is:

$$\frac{\partial f}{\partial y}(t, y) = \frac{1}{1+t}.$$

On D, it holds that $0 \le t \le 1$, so

$$\frac{1}{1+t} \le 1 \quad \text{for all } (t, y) \in D.$$

Thus

$$\left|\frac{\partial f}{\partial y}(t, y)\right| \le 1 \quad \text{on } D.$$

By Theorem 5.3, f satisfies a Lipschitz condition in the variable y on D with Lipschitz constant $L = 1$.

(ii) Applicability of Theorem 5.6

On D, $f(t, y)$ is continuous (it is a rational function with denominator $1 + t \neq 0$ for $t \in [0, 1]$), and it has been shown that f is Lipschitz in y on D.

Therefore, the hypotheses of Theorem 5.6 are satisfied with $a = 0$, $b = 1$, and $\alpha = 1$.

(d) $f(t,y) = \dfrac{y^2}{1+t}$

(i) Lipschitz on D?

The partial derivative is:

$$\frac{\partial f}{\partial y}(t,y) = \frac{2y}{1+t}.$$

On D, $0 \le t \le 1$, so $1 \le 1+t \le 2$. However, y can be any real number, so $|2y/(1+t)|$ can be arbitrarily large as $|y| \to \infty$. Thus $\left|\frac{\partial f}{\partial y}\right|$ is **not** bounded on D, so a global constant L cannot be found such that

$$\left|\frac{\partial f}{\partial y}(t,y)\right| \le L \quad \text{for all } (t,y) \in D.$$

Equivalently, looking directly at the Lipschitz inequality: for fixed t,

$$|f(t,y_1) - f(t,y_2)| = \left|\frac{y_1^2 - y_2^2}{1+t}\right| = \frac{|y_1 - y_2| \cdot |y_1 + y_2|}{1+t}.$$

To have a Lipschitz constant L on all of D, it is required that

$$\frac{|y_1 + y_2|}{1+t} \le L \quad \text{for all } y_1, y_2 \in \mathbb{R},\ 0 \le t \le 1,$$

which is **impossible** since $|y_1 + y_2|$ can be arbitrarily large.

(ii) Applicability of Theorem 5.6

The function $f(t,y) = \dfrac{y^2}{1+t}$ is continuous on D, but Theorem 5.6 **also** requires that f be Lipschitz in y on all of D. Since it has been shown that f is not globally Lipschitz in y on D, the hypotheses of Theorem 5.6 are not satisfied.

Consequently, even though the IVP may still have a unique solution and be well posed by other arguments, **Theorem 5.6 cannot be used to prove this** on the given domain D.

6

This is linear: $y' + y = 1$.

The integrating factor is $\mu(t) = e^t$, so

$$(e^t y)' = e^t.$$

Integrating yields:

$$e^t y(t) = \int_0^t e^s ds + C = (e^t - 1) + C.$$

Using $y(0) = 0$: $0 = (1-1) + C \Rightarrow C = 0$. Hence

$$y(t) = 1 - e^{-t}.$$

The perturbed IVP is

$$z' = 1 - z + \delta t, \quad 0 \le t \le 2, \quad z(0) = \delta_0,$$

where the perturbations satisfy $|\delta| < \varepsilon,\ |\delta_0| < \varepsilon$.

Rewriting as

$$z' + z = 1 + \delta t.$$

Using the integrating factor e^t again:

$$(e^t z)' = e^t(1 + \delta t).$$

Integrating from 0 to t:

$$e^t z(t) - z(0) = \int_0^t e^s(1 + \delta s)ds = \int_0^t e^s ds + \delta \int_0^t s e^s ds.$$

The integrals are computed as follows:

- $\displaystyle\int_0^t e^s ds = e^t - 1.$
- $\displaystyle\int_0^t s e^s ds = [(s-1)e^s]_0^t = (t-1)e^t - (-1)e^0 = (t-1)e^t + 1.$

So

$$e^t z(t) = \delta_0 + (e^t - 1) + \delta\big[(t-1)e^t + 1\big],$$

hence

$$z(t) = e^{-t}\big[\delta_0 + e^t - 1 + \delta\big((t-1)e^t + 1\big)\big] = e^{-t}\delta_0 + (1 - e^{-t}) + \delta\big[(t-1) + e^{-t}\big]$$

Define

$$w(t) := z(t) - y(t) = e^{-t}\delta_0 + \delta\big[(t-1) + e^{-t}\big].$$

Taking absolute values and using $|\delta_0| < \varepsilon,\ |\delta| < \varepsilon$:

$$|w(t)| \le |e^{-t}\delta_0| + |\delta||t - 1 + e^{-t}| \le e^{-t}\varepsilon + \varepsilon|t - 1 + e^{-t}|.$$

For $0 \le t \le 2$:

- $0 \le e^{-t} \le 1$,
- $|t-1| \le 1$, and $e^{-t} \le 1$, so

$$|t - 1 + e^{-t}| \le |t-1| + e^{-t} \le 1 + 1 = 2.$$

Therefore, for all $t \in [0, 2]$,

$$|w(t)| \le \varepsilon(1 + 2) = 3\varepsilon.$$

Choosing, for example, any $\varepsilon_0 > 0$ and $k = 3$, it follows that

$$|z(t) - y(t)| \le 3\varepsilon \quad \text{for all } t \in [0, 2],$$

whenever $|\delta| < \varepsilon$ and $|\delta_0| < \varepsilon$.

Conclusion (well-posedness)

- Existence and uniqueness: $f(t, y) = 1 - y$ is continuous in (t, y) and Lipschitz in y (with Lipschitz constant $L = 1$), so the IVP has a unique solution on $[0, 2]$.
- Continuous dependence on data: for any small perturbations of the differential equation of the form $\delta(t) = \delta t$ and initial condition perturbation δ_0, the resulting solution $z(t)$ satisfies

$$|z(t) - y(t)| \le 3\varepsilon \quad \text{on } [0, 2].$$

Thus the initial-value problem

$$y' = 1 - y, \quad 0 \le t \le 2, \quad y(0) = 0$$

is **well posed**.

10

a. Derivation of Picard's method

Start from the differential equation

$$y'(t) = f(t, y(t)), \quad a \le t \le b, \quad y(a) = \alpha.$$

Integrate both sides from a to t:

$$\int_a^t y'(s)ds = \int_a^t f(s, y(s))ds.$$

The left-hand side is $y(t) - y(a)$, so

$$y(t) - y(a) = \int_a^t f(s, y(s))ds.$$

Using the initial condition $y(a) = \alpha$, the **integral equation** is obtained:

$$y(t) = \alpha + \int_a^t f(s, y(s))ds. \tag{*}$$

Picard's method con structs a sequence of approximations $\{y_k(t)\}$ by replacing the unknown y on the right-hand side with the previous iterate:

- Start with some initial guess, usually

$$y_0(t) = \alpha.$$

- Then define, for $k \ge 1$,

$$y_k(t) = \alpha + \int_a^t f\big(s, y_{k-1}(s)\big)\,ds.$$

This is exactly Picard's method.

b . Picard iterates

Here $a = 0,\ \alpha = 1,\ f(t,y) = -y + t + 1$.

By Picard's method:

$$y_0(t) = 1.$$

Then for $k \geq 1$,

$$y_k(t) = 1 + \int_0^t \big[- y_{k-1}(\tau) + \tau + 1\big]d\tau.$$

$$y_0(t) = 1.$$

$$\begin{aligned} y_1(t) &= 1 + \int_0^t \big[- y_0(\tau) + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big[- 1 + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \tau d\tau \\ &= 1 + \big[\tfrac{1}{2}\tau^2\big]_0^t \\ &= 1 + \tfrac{1}{2}t^2. \end{aligned}$$

$$\begin{aligned} y_2(t) &= 1 + \int_0^t \big[- y_1(\tau) + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big[- \big(1 + \tfrac{1}{2}\tau^2\big) + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big[-1 - \tfrac{1}{2}\tau^2 + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big(\tau - \tfrac{1}{2}\tau^2\big)d\tau \\ &= 1 + \big[\tfrac{1}{2}\tau^2 - \tfrac{1}{6}\tau^3\big]_0^t \\ &= 1 + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3. \end{aligned}$$

$$\begin{aligned} y_3(t) &= 1 + \int_0^t \big[- y_2(\tau) + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big[- \big(1 + \tfrac{1}{2}\tau^2 - \tfrac{1}{6}\tau^3\big) + \tau + 1\big]d\tau \\ &= 1 + \int_0^t \big(-1 - \tfrac{1}{2}\tau^2 + \tfrac{1}{6}\tau^3 + \tau + 1\big)d\tau \\ &= 1 + \int_0^t \big(\tau - \tfrac{1}{2}\tau^2 + \tfrac{1}{6}\tau^3\big)d\tau. \end{aligned}$$

Integrate term-by-term:

$$\int_0^t \tau d\tau = \tfrac{1}{2}t^2, \quad \int_0^t -\tfrac{1}{2}\tau^2 d\tau = -\tfrac{1}{6}t^3, \quad \int_0^t \tfrac{1}{6}\tau^3 d\tau = \tfrac{1}{24}t^4.$$

Thus

$$y_3(t) = 1 + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3 + \tfrac{1}{24}t^4.$$

c. Comparison with the Maclaurin series of the exact solution

The exact solution is

$$y(t) = t + e^{-t}.$$

The Maclaurin series for e^{-t} is

$$e^{-t} = 1 - t + \frac{t^2}{2!} - \frac{t^3}{3!} + \frac{t^4}{4!} - \cdots = 1 - t + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3 + \tfrac{1}{24}t^4 - \cdots$$

So

$$\begin{aligned} y(t) &= t + e^{-t} \\ &= t + \big(1 - t + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3 + \tfrac{1}{24}t^4 - \cdots\big) \\ &= 1 + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3 + \tfrac{1}{24}t^4 - \cdots \end{aligned}$$

Thus the Maclaurin series of the exact solution is

$$y(t) = 1 + \tfrac{1}{2}t^2 - \tfrac{1}{6}t^3 + \tfrac{1}{24}t^4 - \cdots$$

Comparing with the Picard iterates:

- $y_0(t) = 1$: just the constant term.
- $y_1(t) = 1 + \frac{1}{2}t^2$: matches the first two terms of the series.
- $y_2(t) = 1 + \frac{1}{2}t^2 - \frac{1}{6}t^3$: matches up to the t^3 term.
- $y_3(t) = 1 + \frac{1}{2}t^2 - \frac{1}{6}t^3 + \frac{1}{24}t^4$: matches up to the t^4 term.

So each Picard iterate $y_k(t)$ reproduces the Maclaurin series of the exact solution truncated after a certain degree. As k increases, the Picard iterates approximate more terms of the Taylor (Maclaurin) series of the true solution.

Problem 3: Euler's method

2

(a) $y' = e^{t-y}, 0 \le t \le 1, y(0) = 1, h = 0.5$

- $t_0 = 0, ; w_0 = 1$

$$f(t_0, w_0) = e^{0-1} = e^{-1} \approx 0.36788,$$
$$w_1 = w_0 + hf(t_0, w_0) = 1 + 0.5(0.36788) \approx 1.18394.$$

- $t_1 = 0.5, ; w_1 \approx 1.18394$

$$f(t_1, w_1) = e^{0.5 - 1.18394} \approx e^{-0.68394} \approx 0.50462,$$
$$w_2 = w_1 + 0.5(0.50462) \approx 1.18394 + 0.25231 \approx 1.43625.$$

Euler approximations:

t	$w(t)$
0	1.00000
0.5	1.18394
1.0	1.43625

(b) $y' = \dfrac{1+t}{1+y}, ; 1 \le t \le 2, ; y(1) = 2, ; h = 0.5$

- $t_0 = 1, ; w_0 = 2$

$$f(t_0, w_0) = \frac{1+1}{1+2} = \frac{2}{3} \approx 0.66667,$$
$$w_1 = 2 + 0.5\left(\frac{2}{3}\right) = 2 + \frac{1}{3} \approx 2.33333.$$

- $t_1 = 1.5, ; w_1 \approx 2.33333$

$$f(t_1, w_1) = \frac{1 + 1.5}{1 + 2.33333} = \frac{2.5}{3.33333} \approx 0.75,$$
$$w_2 = 2.33333 + 0.5(0.75) = 2.33333 + 0.375 \approx 2.70833.$$

Euler approximations:

t	$w(t)$
1.0	2.00000
1.5	2.33333
2.0	2.70833

(c) $y' = -y + ty^{1/2}, ; 2 \le t \le 3, ; y(2) = 2, ; h = 0.25$

We need $t = 2, 2.25, 2.5, 2.75, 3$.

- $t_0 = 2, ; w_0 = 2$

$$f(t_0, w_0) = -2 + 2\sqrt{2} \approx -2 + 2(1.41421) \approx 0.82842,$$
$$w_1 = 2 + 0.25(0.82842) \approx 2.20711.$$

- $t_1 = 2.25, ; w_1 \approx 2.20711$

$$f(t_1, w_1) = -w_1 + t_1\sqrt{w_1} \approx -2.20711 + 2.25\sqrt{2.20711} \approx 1.13557,$$
$$w_2 = 2.20711 + 0.25(1.13557) \approx 2.49099.$$

- $t_2 = 2.5, ; w_2 \approx 2.49099$

$$f(t_2, w_2) = -w_2 + 2.5\sqrt{w_2} \approx 1.45473,$$
$$w_3 = 2.49099 + 0.25(1.45473) \approx 2.85468.$$

- $t_3 = 2.75, ; w_3 \approx 2.85468$

$$f(t_3, w_3) = -w_3 + 2.75\sqrt{w_3} \approx 1.79166,$$
$$w_4 = 2.85468 + 0.25(1.79166) \approx 3.30260.$$

Euler approximations:

t	$w(t)$
2.00	2.00000
2.25	2.20711
2.50	2.49099
2.75	2.85468
3.00	3.30260

(d) $y' = t^{-2}(\sin 2t - 2ty), ; 1 \le t \le 2, ; y(1) = 2, ; h = 0.25$

So $f(t,y) = \dfrac{\sin 2t - 2ty}{t^2}$, with $t = 1, 1.25, 1.5, 1.75, 2$.

- $t_0 = 1, ; w_0 = 2$

$$f(t_0, w_0) = \sin 2 - 4 \approx 0.90930 - 4 \approx -3.09070,$$
$$w_1 = 2 + 0.25(-3.09070) \approx 1.22732.$$

- $t_1 = 1.25, ; w_1 \approx 1.22732$

$$f(t_1, w_1) = \frac{\sin 2.5 - 2(1.25)w_1}{1.25^2} = \frac{\sin 2.5 - 2.5w_1}{1.5625} \approx -1.58070,$$
$$w_2 = 1.22732 + 0.25(-1.58070) \approx 0.83215.$$

- $t_2 = 1.5, ; w_2 \approx 0.83215$

$$f(t_2, w_2) = \frac{\sin 3 - 3 \cdot 2w_2}{1.5^2} = \frac{\sin 3 - 3w_2}{2.25} \approx -1.04681,$$
$$w_3 = 0.83215 + 0.25(-1.04681) \approx 0.57045.$$

- $t_3 = 1.75, ; w_3 \approx 0.57045$

$$f(t_3, w_3) = \frac{\sin 3.5 - 2(1.75)w_3}{1.75^2} = \frac{\sin 3.5 - 3.5w_3}{3.0625} \approx -0.76648,$$
$$w_4 = 0.57045 + 0.25(-0.76648) \approx 0.37883.$$

Euler approximations:

t	$w(t)$
1.00	2.00000
1.25	1.22732
1.50	0.83215
1.75	0.57045
2.00	0.37883

These $w(t_i)$ are the requested Euler-method approximations for each IVP.

4

(a) $y' = e^{t-y}, ; y(0) = 1, ; h = 0.5, ; y(t) = \ln(e^t + e - 1)$

Along the true solution on $0 \le t \le 1$,

$$y(t) = \ln(e^t + e - 1), \quad 1 \le y(t) \le y(1) = \ln(2e - 1) < 1.5.$$

On the rectangle $D = [0,1] \times [1, 1.5]$,

$$f(t,y) = e^{t-y}, \quad f_y = -e^{t-y}, \quad |f_y| \le e^{1-1} = 1,$$

so take $L = 1$.

From

$$y(t) = \ln(e^t + e - 1),$$

one computes

$$y'(t) = \frac{e^t}{e^t + e - 1}, \qquad y''(t) = \frac{e^t(e-1)}{(e^t + e - 1)^2},$$

and on $0 \le t \le 1$ this satisfies $|y''(t)| \le \frac{1}{4}$, so take $M = \frac{1}{4}$.

Here $a = 0, ; h = 0.5, ; L = 1, ; M = \frac{1}{4}$, so

$$\frac{hM}{2L} = \frac{0.5 \cdot 0.25}{2} = \frac{1}{16} = 0.0625, \quad |y(t_i) - w_i| \le \frac{1}{16}\left(e^{t_i} - 1\right).$$

Numerical comparison

t_i	w_i	$y(t_i)$	**actual error** $\|y-w\|$	**error bound**
0.0	1.000000	1.000000	0.000000	0.000000
0.5	1.183940	1.214023	0.03008	0.04055
1.0	1.436252	1.489880	0.05363	0.10739

So in each case the **actual error is smaller than the error bound**.

(b) $y' = \dfrac{1+t}{1+y}, ; y(1) = 2, ; h = 0.5, ; y(t) = \sqrt{t^2+2t+6} - 1$

On $[1,2]$,

$$y(1) = 2, \quad y(2) = \sqrt{14} - 1 \approx 2.74,$$

so the solution stays in $2 \le y(t) \le 3$. On
$D = [1,2] \times [2,3]$,

$$f(t,y) = \frac{1+t}{1+y}, \quad f_y = -\frac{1+t}{(1+y)^2}.$$

Thus

$$|f_y| \le \frac{1+2}{(1+2)^2} = \frac{3}{9} = \frac{1}{3},$$

so take $L = \frac{1}{3}$.

Write $y(t) = \sqrt{g(t)} - 1$ with $g(t) = t^2 + 2t + 6$. Then

$$y'(t) = \frac{t+1}{\sqrt{g(t)}}, \qquad y''(t) = \frac{5}{[t^2+2t+6]^{3/2}}.$$

On $1 \le t \le 2$, the denominator is minimal at $t = 1$ (where $g(1) = 9$), so

$$|y''(t)| \le \frac{5}{9^{3/2}} = \frac{5}{27},$$

hence $M = \frac{5}{27}$.

Here $a = 1, ; h = 0.5, ; L = \frac{1}{3}, ; M = 5/27$:

$$\frac{hM}{2L} = \frac{0.5 \cdot \frac{5}{27}}{2 \cdot \frac{1}{3}} = \frac{5}{36} \approx 0.1389,$$

so

$$|y(t_i) - w_i| \le \frac{5}{36}\left(e^{(t_i-1)/3} - 1\right).$$

Numerical comparison

t_i	w_i	$y(t_i)$	**actual error** $\|y-w\|$	**error bound**
1.0	2.000000	2.000000	0.00000	0.00000
1.5	2.333333	2.354102	0.02077	0.02519
2.0	2.708333	2.741657	0.03332	0.05495

Again, **actual errors are all below the bound**.

(c) $y' = -y + t\sqrt{y}, ; y(2) = 2, ; h = 0.25, ; y(t) = \left(t - 2 + \sqrt{2}e e^{-t/2}\right)^2$

The exact solution on $[2,3]$ satisfies

$$2 \le y(t) \le y(3) \approx 3.45,$$

so consider $D = [2,3] \times [2, 3.5]$. There

$$f(t,y) = -y + t\sqrt{y}, \quad f_y = -1 + \frac{t}{2\sqrt{y}}.$$

On that rectangle, a direct check shows

$$|f_y(t,y)| \le 0.47 \quad \Rightarrow \quad L \approx 0.47.$$

For simplicity take a slightly larger constant $L = 0.5$.

Writing $y(t) = [h(t)]^2$ with
$h(t) = t - 2 + \sqrt{2}e e^{-t/2}$,
one has $y''(t) = 2[h'(t)]^2 + 2h(t)h''(t)$. Both h, h', h'' are continuous on $[2,3]$, and a numerical evaluation shows

$$|y''(t)| \le 1.45 \quad \text{for } 2 \le t \le 3.$$

Take $M = 1.5$.

Here $a = 2, ; h = 0.25, ; L = 0.5, ; M = 1.5$:

$$\frac{hM}{2L} = \frac{0.25 \cdot 1.5}{2 \cdot 0.5} = 0.375,$$

so

$$|y(t_i) - w_i| \le 0.375\big(e^{0.5(t_i - 2)} - 1\big).$$

Numerical comparison

t_i	w_i	$y(t_i)$	**actual error** $\|y - w\|$	**error bound**
2.00	2.000000	2.000000	~0.00000	0.00000
2.25	2.207107	2.244121	0.03701	0.04993
2.50	2.490999	2.564452	0.07345	0.10651
2.75	2.854680	2.965194	0.11051	0.17062
3.00	3.302596	3.451287	0.14869	0.24327

Once more, **every actual error is below its bound**.

(d) $y' = t^{-2}(\sin 2t - 2ty), ; y(1) = 2, ; h = 0.25, ; y(t) = \dfrac{4 + \cos 2 - \cos 2t}{2t^2}$

Here

$$f(t,y) = \frac{\sin 2t - 2ty}{t^2}, \quad f_y = -\frac{2}{t},$$

so on $1 \le t \le 2$,

$$|f_y| \le 2 \quad \Rightarrow \quad L = 2.$$

Using the explicit formula for $y(t)$ and differentiating twice, one finds that $|y''(t)|$ is largest near $t = 1$ and satisfies

$$|y''(t)| \le 7.6 \quad \text{on } [1,2].$$

Choose the convenient bound $M = 8$.

With $a = 1, ; h = 0.25, ; L = 2, ; M = 8$:

$$\frac{hM}{2L} = \frac{0.25 \cdot 8}{2 \cdot 2} = 0.5,$$

so

$$|y(t_i) - w_i| \le 0.5\big(e^{2(t_i - 1)} - 1\big).$$

Numerical comparison

t_i	w_i	$y(t_i)$	**actual error** $\|y - w\|$	**error bound**
1.00	2.000000	2.000000	0.00000	0.00000

t_i	w_i	$y(t_i)$	actual error $\|y - w\|$	error bound
1.25	1.227324	1.403199	0.17587	0.32436
1.50	0.832150	1.016410	0.18426	0.85914
1.75	0.570447	0.738010	0.16756	1.74084
2.00	0.378827	0.529687	0.15086	3.19453

Again, **all actual errors are significantly smaller than the theoretical bounds**.

10

Let

$$f(t, y) = \frac{1}{t^2} - \frac{y}{t} - y^2, \quad h = 0.05, \quad t_i = 1 + 0.05i.$$

Euler's method:

$$w_0 = -1, \qquad w_{i+1} = w_i + hf(t_i, w_i).$$

(a) Euler's method with $h = 0.05$ & (b) Linear interpolation using the Euler points

```
%% MATH 128A - Euler's Method + Linear Interpolation
% IVP: y' = 1/t^2 - y/t - y^2,  1 <= t <= 2,  y(1) = -1
% Exact solution: y(t) = -1/t

clear; clc;

%% Part (a): Euler's method with h = 0.05

% Parameters
a = 1;          % left endpoint
b = 2;          % right endpoint
h = 0.05;       % step size
N = (b - a) / h;

% Preallocate arrays
t = a + (0:N)*h;          % mesh points t_i
w = zeros(1, N+1);        % Euler approximations w_i

% Initial condition
w(1) = -1;                % y(1) = -1

% Define f(t, y)
f = @(t, y) 1./t.^2 - y./t - y.^2;

% Euler iteration
for i = 1:N
    w(i+1) = w(i) + h * f(t(i), w(i));
end

% Exact solution at mesh points
y_exact = -1 ./ t;

% Error at mesh points
err = abs(y_exact - w);

% Display table for part (a)
fprintf('Part (a): Euler method results (h = %.4f)\n', h);
fprintf('-------------------------------------------------------------------\n');
fprintf('  i     t_i        w_i (Euler)      y(t_i) exact     |error|\n');
fprintf('-------------------------------------------------------------------\n');
for i = 1:N+1
    fprintf('%3d   %6.3f    %14.6f   %14.6f   %10.6f\n', ...
            i-1, t(i), w(i), y_exact(i), err(i));
end
fprintf('-------------------------------------------------------------------\n\n');

%% Part (b): Linear interpolation at off-mesh points

% Query points
t_query = [1.052, 1.555, 1.978];

fprintf('Part (b): Linear interpolation using Euler points\n');
fprintf('===================================================================\n');
fprintf('  t        y_interp (linear)    y_exact = -1/t      |y_interp - y_exact|\n');
fprintf('===================================================================\n');
```

```
for k = 1:length(t_query)
    tq = t_query(k);

    % Find index i such that t(i) <= tq <= t(i+1)
    i = find(t <= tq, 1, 'last');
    if i == length(t)
        error('Query point %.3f is outside the interpolation range.', tq);
    end

    % Linear interpolation between (t(i), w(i)) and (t(i+1), w(i+1))
    theta = (tq - t(i)) / (t(i+1) - t(i));       % in [0,1]
    y_interp = w(i) + theta * (w(i+1) - w(i));

    % Exact value and error
    y_true = -1 / tq;
    err_q  = abs(y_interp - y_true);

    fprintf('%7.3f   %16.6f   %16.6f   %16.6f\n', tq, y_interp, y_true, err_q);
end

fprintf('=========================================================================\n');
```

Outputs:

```
Part (a): Euler method results (h = 0.0500)
-----------------------------------------------------------------
  i     t_i        w_i (Euler)      y(t_i) exact     |error|
-----------------------------------------------------------------
  0    1.000         -1.000000        -1.000000     0.000000
  1    1.050         -0.950000        -0.952381     0.002381
  2    1.100         -0.904535        -0.909091     0.004555
  3    1.150         -0.863007        -0.869565     0.006558
  4    1.200         -0.824917        -0.833333     0.008416
  5    1.250         -0.789848        -0.800000     0.010152
  6    1.300         -0.757447        -0.769231     0.011784
  7    1.350         -0.727415        -0.740741     0.013326
  8    1.400         -0.699495        -0.714286     0.014791
  9    1.450         -0.673467        -0.689655     0.016188
 10    1.500         -0.649141        -0.666667     0.017525
 11    1.550         -0.626350        -0.645161     0.018811
 12    1.600         -0.604949        -0.625000     0.020051
 13    1.650         -0.584812        -0.606061     0.021249
 14    1.700         -0.565825        -0.588235     0.022410
 15    1.750         -0.547890        -0.571429     0.023539
 16    1.800         -0.530918        -0.555556     0.024637
 17    1.850         -0.514832        -0.540541     0.025708
 18    1.900         -0.499561        -0.526316     0.026754
 19    1.950         -0.485043        -0.512821     0.027778
 20    2.000         -0.471220        -0.500000     0.028780
-----------------------------------------------------------------

Part (b): Linear interpolation using Euler points
=========================================================================
  t        y_interp (linear)    y_exact = -1/t      |y_interp - y_exact|
=========================================================================
  1.052          -0.948181          -0.950570           0.002389
  1.555          -0.624210          -0.643087           0.018877
  1.978          -0.477302          -0.505561           0.028259
=========================================================================
```

(c) Step size h so that $|y(t_i) - w_i| \leq 0.05$

Use Eq. (5.10):

$$|y(t_i) - w_i| \leq \frac{hM}{2L}\left(e^{L(t_i-a)} - 1\right),$$

where L is a Lipschitz constant for f in y on a region D, and
M bounds $|y''(t)|$ on $[a, b]$.

Take

$$D = [1, 2] \times [-1.05, -0.45].$$

$$f(t, y) = \frac{1}{t^2} - \frac{y}{t} - y^2, \quad \frac{\partial f}{\partial y}(t, y) = -\frac{1}{t} - 2y.$$

On D: $1 \leq t \leq 2$, $-1.05 \leq y \leq -0.45$.

The maximum of $|-1/t - 2y|$ on D occurs at the corner $t = 2,\ y = -1.05$:

$$\frac{\partial f}{\partial y}(2,-1.05) = -\frac{1}{2} - 2(-1.05) = -\frac{1}{2} + 2.1 = 1.6.$$

At other corners we get smaller magnitudes (e.g. -0.1, 0.4, 1.1), so

$$\left|\frac{\partial f}{\partial y}(t,y)\right| \le 1.6 \quad \text{for all } (t,y) \in D.$$

Thus we can take

$$L = 1.6.$$

Because D is a rectangle, it is convex, so by Theorem 5.3 this gives a Lipschitz constant on D.

We know the exact solution:

$$y(t) = -\frac{1}{t}, \quad y'(t) = \frac{1}{t^2}, \quad y''(t) = -\frac{2}{t^3}.$$

Hence, on $[1,2]$,

$$|y''(t)| = \frac{2}{t^3} \le 2,$$

so we may take

$$M = 2.$$

Take $a = 1$, $b = 2$. The worst case in Eq. (5.10) occurs at $t_i = 2$:

$$|y(2) - w_N| \le \frac{hM}{2L}\left(e^{L(2-1)} - 1\right) = \frac{h \cdot 2}{2 \cdot 1.6}\left(e^{1.6} - 1\right) = \frac{h}{1.6}\left(e^{1.6} - 1\right).$$

We want this ≤ 0.05:

$$\frac{h}{1.6}\left(e^{1.6} - 1\right) \le 0.05 \quad \Longrightarrow \quad h \le \frac{0.05 \cdot 1.6}{e^{1.6} - 1}.$$

Numerically,

$$e^{1.6} \approx 4.95, \quad e^{1.6} - 1 \approx 3.95,$$

so

$$h \lesssim \frac{0.08}{3.95} \approx 0.0202.$$

Thus a sufficient condition is

$$\boxed{h \le 0.0202 \text{ (approximately)}}$$

to guarantee $|y(t_i) - w_i| \le 0.05$ for $1 \le t_i \le 2$ by Eq. (5.10).

Justification for using a rectangle D

- The exact solution:

  $$y(t) = -\frac{1}{t}, \quad t \in [1,2] \;\Rightarrow\; y(t) \in [-1, -0.5],$$

  which lies strictly inside $[-1.05, -0.45]$.
- If we choose h so that $|y(t_i) - w_i| \le 0.05$, then each Euler approximation satisfies

  $$w_i \in [y(t_i) - 0.05, y(t_i) + 0.05] \subset [-1.05, -0.45],$$

  so **both** the exact solution and the numerical solution stay inside the rectangle D.

On this rectangle, f is continuous and satisfies a Lipschitz condition in y with constant $L = 1.6$, and $|y''(t)| \le M = 2$. The proof of Theorem 5.9 only needs these properties on a convex set containing the trajectories, so it still applies when D is the rectangle $[1,2] \times [-1.05, -0.45]$. Hence Eq. (5.10) is valid here, and the above bound on h is justified.

12

What happens with Euler's method, $h = 0.1$?

With $h = 0.1$,

$$w_{n+1} = w_n + 0.1(-10 w_n) = w_n - w_n = 0.$$

Thus the recurrence is

$$w_{n+1} = 0 \cdot w_n.$$

With $w_0 = 1$:

- $w_1 = 0$,
- $w_2 = 0$,
- $\dots$
- $w_n = 0$ for all $n \geq 1$.

So the Euler approximation is:

$$w(t_0) = 1 \text{ at } t_0 = 0, \quad w(t_n) = 0 \text{ at } t_n = 0.1n \text{ for } n \geq 1.$$

However, the true solution at $t = 0.1$ is

$$y(0.1) = e^{-1} \approx 0.3679,$$

so the method "over-damps" the solution and instantly drops it to zero.

Does this behavior violate Theorem 5.9?

No, it does **not** violate Theorem 5.9.

The hypotheses are verified as follows:

- $f(t, y) = -10y$ is continuous in (t, y) and satisfies a Lipschitz condition in y on any strip $D = \{(t, y) : 0 \leq t \leq 2, \ -\infty < y < \infty\}$, with
$$\frac{\partial f}{\partial y} = -10 \Rightarrow L = 10.$$
- The exact solution is $y(t) = e^{-10t}$, so
$$y''(t) = 100e^{-10t},$$
hence $|y''(t)| \leq 100$ on $[0, 2]$, so $M = 100$ can be taken.

Theorem 5.9 gives the error bound

$$|y(t_i) - w_i| \leq \frac{hM}{2L}\left(e^{L(t_i - a)} - 1\right) = \frac{0.1 \cdot 100}{2 \cdot 10}\left(e^{10t_i} - 1\right) = 0.5\left(e^{10t_i} - 1\right).$$

At the first step, $t_1 = 0.1$:

- Euler approximation: $w_1 = 0$.
- Exact solution: $y(0.1) = e^{-1} \approx 0.3679$.
- Actual error: $|y(0.1) - w_1| = e^{-1} \approx 0.3679$.
- Bound from Theorem 5.9:
$$0.5(e^1 - 1) \approx 0.5(2.71828 - 1) \approx 0.859,$$
which is **larger** than the actual error, so the inequality holds.

For later t_i, the bound $0.5(e^{10t_i} - 1)$ becomes very large, but this simply indicates that the theorem provides a conservative (loose) upper bound; it does **not** imply that the error must be close to that bound, only that it cannot exceed it.

In conclusion:

- Euler's method with $h = 0.1$ produces a sequence that jumps to zero and stays there.
- This is a qualitatively poor approximation of the true solution, but it **still satisfies** the error inequality of Theorem 5.9.
- Therefore, **there is no violation** of Theorem 5.9; the theorem's bound remains valid.

Problem 4: Runge-Kutta methods

2 & 6 & 14

MATLAB codes:

```
function solve_all_methods
% SOLVE_ALL_METHODS
%   Solve the four IVPs using Modified Euler, Midpoint, and RK4,
%   and compare with the exact solutions at the mesh points.
%
%   Problems:
%   (a) y' = e^{t - y},                  0 <= t <= 1,  y(0) = 1,    h = 0.5
%       exact: y(t) = ln(e^t + e - 1)
%
```

```
%   (b) y' = (1 + t)/(1 + y),            1 <= t <= 2,  y(1) = 2,    h = 0.5
%       exact: y(t) = sqrt(t^2 + 2t + 6) - 1
%
%   (c) y' = -y + t*sqrt(y),             2 <= t <= 3,  y(2) = 2,    h = 0.25
%       exact: y(t) = (t - 2 + sqrt(2)*exp(1)*exp(-t/2)).^2
%
%   (d) y' = t^{-2} (sin(2t) - 2 t y),   1 <= t <= 2,  y(1) = 2,    h = 0.25
%       exact: y(t) = (4 + cos(2) - cos(2t)) ./ (2*t.^2)

    clc;

    %% Problem (a)
    fprintf('============================\n');
    fprintf('Problem (a): y'' = e^{t-y}, 0 <= t <= 1, y(0) = 1, h = 0.5\n');
    fa   = @(t,y) exp(t - y);
    ya   = @(t) log(exp(t) + exp(1) - 1);  % exact solution
    a_a  = 0; b_a = 1; alpha_a = 1; h_a = 0.5;

    solve_and_print(fa, ya, a_a, b_a, alpha_a, h_a, 'Modified Euler', @step_modified_euler);
    solve_and_print(fa, ya, a_a, b_a, alpha_a, h_a, 'Midpoint',       @step_midpoint);
    solve_and_print(fa, ya, a_a, b_a, alpha_a, h_a, 'RK4',            @step_rk4);

    %% Problem (b)
    fprintf('\n============================\n');
    fprintf('Problem (b): y'' = (1+t)/(1+y), 1 <= t <= 2, y(1) = 2, h = 0.5\n');
    fb   = @(t,y) (1 + t) ./ (1 + y);
    yb   = @(t) sqrt(t.^2 + 2*t + 6) - 1;  % exact solution
    a_b  = 1; b_b = 2; alpha_b = 2; h_b = 0.5;

    solve_and_print(fb, yb, a_b, b_b, alpha_b, h_b, 'Modified Euler', @step_modified_euler);
    solve_and_print(fb, yb, a_b, b_b, alpha_b, h_b, 'Midpoint',       @step_midpoint);
    solve_and_print(fb, yb, a_b, b_b, alpha_b, h_b, 'RK4',            @step_rk4);

    %% Problem (c)
    fprintf('\n============================\n');
    fprintf('Problem (c): y'' = -y + t*sqrt(y), 2 <= t <= 3, y(2) = 2, h = 0.25\n');
    fc   = @(t,y) -y + t.*sqrt(y);
    yc   = @(t) (t - 2 + sqrt(2)*exp(1).*exp(-t/2)).^2;  % exact solution
    a_c  = 2; b_c = 3; alpha_c = 2; h_c = 0.25;

    solve_and_print(fc, yc, a_c, b_c, alpha_c, h_c, 'Modified Euler', @step_modified_euler);
    solve_and_print(fc, yc, a_c, b_c, alpha_c, h_c, 'Midpoint',       @step_midpoint);
    solve_and_print(fc, yc, a_c, b_c, a_c, h_c, 'RK4',                @step_rk4); %#ok<*NASGU>

    %% Problem (d)
    fprintf('\n============================\n');
    fprintf('Problem (d): y'' = t^{-2}(sin(2t) - 2ty), 1 <= t <= 2, y(1) = 2, h = 0.25\n');
    fd   = @(t,y) (sin(2*t) - 2*t.*y) ./ (t.^2);
    yd   = @(t) (4 + cos(2) - cos(2*t)) ./ (2*t.^2);  % exact solution
    a_d  = 1; b_d = 2; alpha_d = 2; h_d = 0.25;

    solve_and_print(fd, yd, a_d, b_d, alpha_d, h_d, 'Modified Euler', @step_modified_euler);
    solve_and_print(fd, yd, a_d, b_d, alpha_d, h_d, 'Midpoint',       @step_midpoint);
    solve_and_print(fd, yd, a_d, b_d, alpha_d, h_d, 'RK4',            @step_rk4);

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Generic driver: solve and print table
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function solve_and_print(f, y_exact, a, b, alpha, h, method_name, step_fun)
    % Number of steps
    N = round((b - a)/h);

    % Preallocate
    t = zeros(N+1,1);
    w = zeros(N+1,1);

    % Initial condition
    t(1) = a;
    w(1) = alpha;

    % March forward using the chosen one-step method
    for i = 1:N
        t(i+1) = a + i*h;
        w(i+1) = step_fun(f, t(i), w(i), h);
    end

    % Exact values and errors
    y = y_exact(t);
    err = y - w;

    % Print table
    fprintf('\nMethod: %s\n', method_name);
```

```
    fprintf('   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w\n');
    fprintf('-------------------------------------------------------------------------------\n');
    for i = 1:N+1
        fprintf('%4d   %10.6f   %16.6f   %16.6f   %14.6e\n', ...
                i-1, t(i), w(i), y(i), err(i));
    end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% One-step methods
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function w_next = step_modified_euler(f, t, w, h)
% MODIFIED EULER (Improved Euler / Explicit Trapezoid)
%   Predictor: w_tilde = w + h*f(t,w)
%   Corrector: w_next  = w + h/2 * ( f(t,w) + f(t+h,w_tilde) )
    k1 = f(t, w);
    w_tilde = w + h*k1;
    k2 = f(t + h, w_tilde);
    w_next = w + (h/2)*(k1 + k2);
end

function w_next = step_midpoint(f, t, w, h)
% MIDPOINT METHOD
%   w_mid  = w + (h/2)*f(t,w)
%   w_next = w + h*f(t + h/2, w_mid)
    k1 = f(t, w);
    w_mid = w + (h/2)*k1;
    k2 = f(t + h/2, w_mid);
    w_next = w + h*k2;
end

function w_next = step_rk4(f, t, w, h)
% CLASSICAL RUNGE-KUTTA METHOD OF ORDER 4
%   k1 = h f(t, w)
%   k2 = h f(t + h/2, w + k1/2)
%   k3 = h f(t + h/2, w + k2/2)
%   k4 = h f(t + h,   w + k3)
%   w_next = w + (1/6) (k1 + 2k2 + 2k3 + k4)
    k1 = h*f(t, w);
    k2 = h*f(t + h/2, w + k1/2);
    k3 = h*f(t + h/2, w + k2/2);
    k4 = h*f(t + h,   w + k3);
    w_next = w + (k1 + 2*k2 + 2*k3 + k4)/6;
end
```

Outputs:

```
============================
Problem (a): y' = e^{t-y}, 0 <= t <= 1, y(0) = 1, h = 0.5

Method: Modified Euler
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     0.000000           1.000000           1.000000     0.000000e+00
   1     0.500000           1.218126           1.214023    -4.103047e-03
   2     1.000000           1.497555           1.489880    -7.674376e-03

Method: Midpoint
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     0.000000           1.000000           1.000000     0.000000e+00
   1     0.500000           1.215430           1.214023    -1.407415e-03
   2     1.000000           1.493239           1.489880    -3.358906e-03

Method: RK4
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     0.000000           1.000000           1.000000     0.000000e+00
   1     0.500000           1.214041           1.214023    -1.788643e-05
   2     1.000000           1.489921           1.489880    -4.116005e-05

============================
Problem (b): y' = (1+t)/(1+y), 1 <= t <= 2, y(1) = 2, h = 0.5

Method: Modified Euler
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.500000           2.354167           2.354102    -6.470042e-05
   2     2.000000           2.741745           2.741657    -8.769601e-05

Method: Midpoint
```

```
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.500000           2.355263           2.354102    -1.161192e-03
   2     2.000000           2.743513           2.741657    -1.855207e-03

Method: RK4
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.500000           2.354103           2.354102    -1.261829e-06
   2     2.000000           2.741659           2.741657    -1.698788e-06

============================
Problem (c): y' = -y + t*sqrt(y), 2 <= t <= 3, y(2) = 2, h = 0.25

Method: Modified Euler
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     2.000000           2.000000           2.000000     4.440892e-16
   1     2.250000           2.245499           2.244121    -1.378344e-03
   2     2.500000           2.567156           2.564452    -2.704017e-03
   3     2.750000           2.969195           2.965194    -4.000684e-03
   4     3.000000           3.456568           3.451287    -5.281770e-03

Method: Midpoint
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     2.000000           2.000000           2.000000     4.440892e-16
   1     2.250000           2.244617           2.244121    -4.959881e-04
   2     2.500000           2.565298           2.564452    -8.460204e-04
   3     2.750000           2.966318           2.965194    -1.123920e-03
   4     3.000000           3.452665           3.451287    -1.378176e-03

Method: RK4
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     2.000000           2.000000           2.000000     4.440892e-16
   1     2.250000           2.244119           2.244121     1.695741e-06
   2     2.500000           2.564449           2.564452     3.171691e-06
   3     2.750000           2.965189           2.965194     4.442195e-06
   4     3.000000           3.451281           3.451287     5.518090e-06

============================
Problem (d): y' = t^{-2}(sin(2t) - 2ty), 1 <= t <= 2, y(1) = 2, h = 0.25

Method: Modified Euler
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.250000           1.416075           1.403199    -1.287611e-02
   2     1.500000           1.031011           1.016410    -1.460092e-02
   3     1.750000           0.752267           0.738010    -1.425691e-02
   4     2.000000           0.543245           0.529687    -1.355790e-02

Method: Midpoint
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.250000           1.436510           1.403199    -3.331131e-02
   2     1.500000           1.051674           1.016410    -3.526370e-02
   3     1.750000           0.769358           0.738010    -3.134848e-02
   4     2.000000           0.556679           0.529687    -2.699153e-02

Method: RK4
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
--------------------------------------------------------------------------------
   0     1.000000           2.000000           2.000000     0.000000e+00
   1     1.250000           1.403357           1.403199    -1.576461e-04
   2     1.500000           1.016559           1.016410    -1.484392e-04
   3     1.750000           0.738132           0.738010    -1.219121e-04
   4     2.000000           0.529786           0.529687    -9.846674e-05
```

29

Let the mesh be defined as $t_i = ih$, and denote the numerical approximation by $w_i \approx y(t_i)$.

Midpoint method

The Midpoint method is given by:

$$w_{i+1} = w_i + hf\left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)\right).$$

Given

$$f(t_i, w_i) = -w_i + t_i + 1,$$

the “midpoint” value is calculated as:

$$y^* = w_i + \frac{h}{2} f(t_i, w_i) = w_i + \frac{h}{2}\big(-w_i + t_i + 1\big).$$

Then, evaluating the function at the midpoint:

$$\begin{aligned} f\left(t_i + \frac{h}{2}, y^*\right) &= -y^* + \left(t_i + \frac{h}{2}\right) + 1 \\ &= -\left[w_i + \frac{h}{2}(-w_i + t_i + 1)\right] + t_i + \frac{h}{2} + 1 \\ &= -w_i + \frac{h}{2} w_i - \frac{h}{2} t_i - \frac{h}{2} + t_i + \frac{h}{2} + 1 \\ &= \left(-1 + \frac{h}{2}\right) w_i + \left(1 - \frac{h}{2}\right) t_i + 1. \end{aligned}$$

Thus, the update becomes:

$$\begin{aligned} w_{i+1} &= w_i + h\left[\left(-1 + \frac{h}{2}\right) w_i + \left(1 - \frac{h}{2}\right) t_i + 1\right] \\ &= w_i\Big(1 - h + \frac{h^2}{2}\Big) + h\Big(1 - \frac{h}{2}\Big) t_i + h. \end{aligned}$$

The Midpoint method yields the recurrence:

$$\boxed{w_{i+1} = \Big(1 - h + \frac{h^2}{2}\Big) w_i + h\Big(1 - \frac{h}{2}\Big) t_i + h.} \tag{M}$$

Modified Euler method

The Modified Euler method is given by:

$$w_{i+1} = w_i + \frac{h}{2}\Big[f(t_i, w_i) + f\big(t_{i+1}, w_i + h f(t_i, w_i)\big)\Big],$$

where again $f(t, y) = -y + t + 1$.

Note that

$$f(t_i, w_i) = -w_i + t_i + 1.$$

First, the Euler predictor is formed:

$$\tilde{y} = w_i + h f(t_i, w_i) = w_i + h(-w_i + t_i + 1) = (1 - h) w_i + h t_i + h.$$

Then f is evaluated at $(t_{i+1}, \tilde{y})$, where $t_{i+1} = t_i + h$:

$$\begin{aligned} f(t_{i+1}, \tilde{y}) &= -\tilde{y} + t_{i+1} + 1 \\ &= -\big[(1 - h) w_i + h t_i + h\big] + (t_i + h) + 1 \\ &= -(1 - h) w_i - h t_i - h + t_i + h + 1 \\ &= -(1 - h) w_i + (1 - h) t_i + 1. \end{aligned}$$

The average of the initial and final slopes is:

$$\begin{aligned} f(t_i, w_i) + f(t_{i+1}, \tilde{y}) &= (-w_i + t_i + 1) + \big(-(1 - h) w_i + (1 - h) t_i + 1\big) \\ &= (2 - h)(-w_i + t_i) + 2. \end{aligned}$$

So the update is:

$$\begin{aligned} w_{i+1} &= w_i + \frac{h}{2}[(2 - h)(-w_i + t_i) + 2] \\ &= w_i + \left(h - \frac{h^2}{2}\right)(-w_i + t_i) + h \\ &= w_i - \left(h - \frac{h^2}{2}\right) w_i + \left(h - \frac{h^2}{2}\right) t_i + h \\ &= \Big(1 - h + \frac{h^2}{2}\Big) w_i + h\Big(1 - \frac{h}{2}\Big) t_i + h. \end{aligned}$$

Thus, the Modified Euler method yields **exactly** the same recurrence:

$$\boxed{w_{i+1} = \Big(1 - h + \frac{h^2}{2}\Big) w_i + h\Big(1 - \frac{h}{2}\Big) t_i + h.} \tag{E}$$

Comparing (M) and (E), it is observed that for any step size h and any step i, the same w_{i+1} is produced from the same w_i. Since both methods start from the same initial value $w_0 = y(0) = 1$, by induction they generate the same sequence $\{w_i\}$ for **all** steps.

Therefore, the Midpoint and Modified Euler methods give the same approximations for this IVP for any choice of h.

Conceptual explanation

The function $f(t, y) = -y + t + 1$ is **affine linear in** y and also linear in t. For such a linear right-hand side,

- the “midpoint slope” used by the Midpoint method,
- and the “average of initial and final slopes” used by the Modified Euler method,

are equivalent linear combinations of w_i and t_i. Consequently, when f is linear, the sampling and combination of slopes in these two Runge–Kutta order-2 schemes become algebraically identical, leading to coincident one-step formulas.

For a general nonlinear $f(t, y)$, the two methods do **not** agree in general; this coincidence relies on the special linear structure of the differential equation $y' = -y + t + 1$.

30

Consider the IVP

$$y' = g(y), \qquad y(t_i) = y_i.$$

The exact solution at $t_{i+1} = t_i + h$ has the Taylor expansion

$$y(t_i + h) = y_i + hy'(t_i) + \frac{h^2}{2}y''(t_i) + \frac{h^3}{6}y'''(t_i) + O(h^4).$$

Using $y' = g(y)$ and the chain rule:

- First derivative:

$$y'(t) = g(y(t)).$$

- Second derivative:

$$y''(t) = \frac{d}{dt}g(y(t)) = g'(y(t))y'(t) = g'(y(t))g(y(t)).$$

  So at $t = t_i$:

$$y''(t_i) = g'(y_i)g(y_i).$$

- Third derivative:

$$y'''(t) = \frac{d}{dt}\big(g'(y(t))g(y(t))\big).$$

  Using product and chain rules:

$$y'''(t) = g''(y(t))y'(t)g(y(t)) + g'(y(t))y''(t).$$

  Substituting $y'(t) = g(y(t))$ and $y''(t) = g'(y(t))g(y(t))$:

$$y'''(t) = g''(y(t))[g(y(t))]^2 + [g'(y(t))]^2 g(y(t)).$$

  Thus at $t = t_i$,

$$y'''(t_i) = g''(y_i)g(y_i)^2 + [g'(y_i)]^2 g(y_i).$$

Therefore:

$$\boxed{y(t_i + h) = y_i + hg(y_i) + \frac{h^2}{2}g'(y_i)g(y_i) + \frac{h^3}{6}\big(g''(y_i)g(y_i)^2 + [g'(y_i)]^2 g(y_i)\big) + O(h^4).}$$

Now applying the *numerical method* starting from the exact value $w_i = y(t_i) = y_i$:

$$w_{i+1} = y_i + ha_1 g(y_i) + ha_2 g\big(y_i + h\delta_2 g(y_i)\big),$$

since in the autonomous case $f(t_i + h\alpha_2, \cdot) = g(\cdot)$, and the time shift doesn’t affect the value of g.

Expanding the inner $g(\cdot)$ in Taylor series about y_i:

$$g\big(y_i + h\delta_2 g(y_i)\big) = g(y_i) + h\delta_2 g'(y_i)g(y_i) + \frac{h^2}{2}\delta_2^2 g''(y_i)g(y_i)^2 + O(h^3).$$

Plugging this back into the method:

$$\begin{aligned} w_{i+1} &= y_i + ha_1 g(y_i) + ha_2\left[g(y_i) + h\delta_2 g'(y_i)g(y_i) + \frac{h^2}{2}\delta_2^2 g''(y_i)g(y_i)^2 + O(h^3)\right] \\ &= y_i + h(a_1 + a_2)g(y_i) + h^2 a_2\delta_2 g'(y_i)g(y_i) + h^3\frac{a_2\delta_2^2}{2}g''(y_i)g(y_i)^2 + O(h^4). \end{aligned}$$

So

$$\boxed{w_{i+1} = y_i + h(a_1 + a_2)g(y_i) + h^2 a_2 \delta_2 g'(y_i)g(y_i) + h^3 \frac{a_2 \delta_2^2}{2} g''(y_i)g(y_i)^2 + O(h^4).}$$

For *local truncation error* to be $O(h^3)$ (meaning the method is order 2), the exact expansion and the method expansion must match **up to and including all h^2 terms**. That gives the usual order-2 conditions:

1. **Coefficient of h:**
$$a_1 + a_2 = 1.$$
2. **Coefficient of $h^2 g'(y_i)g(y_i)$:**
$$a_2 \delta_2 = \frac{1}{2}.$$

If these two equations fail, even the $O(h^2)$ term would be wrong, so the local truncation error would be only $O(h^2)$ (order 1 method).

Assume we choose a_1, a_2, δ_2 so that these two conditions hold. Then both methods agree up to h^2.

Now compare the h^3 terms to check for order 3.

- **Exact solution $y(t_i + h)$:**
$$h^3 \frac{1}{6} g''(y_i)g(y_i)^2 + h^3 \frac{1}{6}[g'(y_i)]^2 g(y_i).$$
- **Method w_{i+1}:**
$$h^3 \frac{a_2 \delta_2^2}{2} g''(y_i)g(y_i)^2.$$

So the difference (local truncation error starting from the exact value) at order h^3 is

$$\begin{aligned}
\tau_i &= y(t_i + h) - w_{i+1} \\
&= h^3 \left[\frac{1}{6} g''(y_i)g(y_i)^2 + \frac{1}{6}[g'(y_i)]^2 g(y_i) - \frac{a_2 \delta_2^2}{2} g''(y_i)g(y_i)^2 \right] + O(h^4) \\
&= h^3 \left[\left(\frac{1}{6} - \frac{a_2 \delta_2^2}{2} \right) g''(y_i)g(y_i)^2 + \frac{1}{6}[g'(y_i)]^2 g(y_i) \right] + O(h^4).
\end{aligned}$$

We could choose constants to fix the $g''g^2$ coefficient, namely enforce

$$\frac{1}{6} - \frac{a_2 \delta_2^2}{2} = 0 \quad \Longrightarrow \quad a_2 \delta_2^2 = \frac{1}{3}.$$

But **no choice of constants** can remove the term

$$\frac{1}{6}[g'(y_i)]^2 g(y_i),$$

because:

- It **does not appear at all** in the method's expansion (the method only generates a $g''g^2$ term at order h^3).
- It depends on the function g itself. For a general nonlinear g with $g(y_i) \neq 0$ and $g'(y_i) \neq 0$, this coefficient is nonzero.

Thus, for a generic autonomous ODE $y' = g(y)$, the local truncation error has an uncancelled term of size

$$\frac{h^3}{6}[g'(y_i)]^2 g(y_i),$$

so the local truncation error is **not** $O(h^4)$; it is only $O(h^3)$.

Equivalently: the local truncation error cannot be smaller than a constant times h^3, so this method can be at best **second order**, not third.

Conclusion

We have shown:

- Matching the h and h^2 terms gives the usual second-order conditions:
$$a_1 + a_2 = 1, \quad a_2 \delta_2 = \frac{1}{2}.$$
- Even after satisfying these, at order h^3 the exact solution has a term involving $[g'(y_i)]^2 g(y_i)$ that **cannot** be reproduced by any choice of the four constants in
$$w_{i+1} = w_i + h a_1 g(w_i) + h a_2 g(w_i + h \delta_2 g(w_i)).$$

Therefore, **no choice of constants** $a_1, a_2, \alpha_2, \delta_2$ can make the local truncation error smaller than order h^3.

So this two-stage method **cannot** be of order three; its best possible local truncation error is $O(h^3)$, i.e. it is at most a **second-order Runge–Kutta method**.

32

The given formula is recognized as the standard RK4 method, and the constants are determined by matching terms.

The classical Runge–Kutta method of order four is

$$\begin{aligned} k_1 &= hf(t_i, w_i), \\ k_2 &= hf\Big(t_i + \tfrac{h}{2}, w_i + \tfrac{1}{2}k_1\Big), \\ k_3 &= hf\Big(t_i + \tfrac{h}{2}, w_i + \tfrac{1}{2}k_2\Big), \\ k_4 &= hf\big(t_i + h, w_i + k_3\big), \\ w_{i+1} &= w_i + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Rewrite w_{i+1} as

$$w_{i+1} = w_i + \frac{h}{6}f(t_i, w_i) + \frac{h}{3}f(\dots) + \frac{h}{3}f(\dots) + \frac{h}{6}f(\dots),$$

matching the terms as follows:

- $\frac{h}{6}f(t_i, w_i)$ corresponds to $\frac{1}{6}k_1$.
- $\frac{h}{3}f(\cdots)$ corresponds to $\frac{2}{6}k_2$.
- The other $\frac{h}{3}f(\cdots)$ corresponds to $\frac{2}{6}k_3$.
- The last $\frac{h}{6}f(\cdots)$ corresponds to $\frac{1}{6}k_4$.

Now match each inner argument.

Second term: matching k_2

Given term:

$$\frac{h}{3}f\big(t_i + \alpha_1 h, w_i + \delta_1 h f(t_i, w_i)\big).$$

We need this to be $\frac{2}{6}k_2 = \frac{h}{3}f\big(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i, w_i)\big)$.

Hence

$$\alpha_1 = \frac{1}{2}, \qquad \delta_1 = \frac{1}{2}.$$

Third term: matching k_3

Given term:

$$\frac{h}{3}f\Big(t_i + \alpha_2 h, w_i + \delta_2 h f\big(t_i + \gamma_2 h, w_i + \gamma_3 h f(t_i, w_i)\big)\Big).$$

We want the inner $f(\cdots)$ to be the one used in k_2, i.e.,

$$f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i, w_i)\right).$$

So set

$$\gamma_2 = \frac{1}{2}, \qquad \gamma_3 = \frac{1}{2}.$$

Then $f(t_i + \gamma_2 h, w_i + \gamma_3 h f(t_i, w_i)) = k_2/h$.

Now we want the *outer* f to produce k_3/h, i.e.,

$$f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right) = f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i, w_i)\right)\right).$$

Inside the outer argument we have

$$w_i + \delta_2 h \cdot f(\text{inner}) = w_i + \delta_2 k_2,$$

so for this to be $w_i + \frac{1}{2}k_2$, we need $\delta_2 = \frac{1}{2}$.

We also need the outer time to be $t_i + h/2$, so $\alpha_2 = \frac{1}{2}$.

Thus

$$\alpha_2 = \frac{1}{2}, \quad \delta_2 = \frac{1}{2}, \quad \gamma_2 = \frac{1}{2}, \quad \gamma_3 = \frac{1}{2}.$$

Fourth term: matching k_4

Given term:

$$\frac{h}{6} f\Big(t_i + \alpha_3 h, w_i + \delta_3 h f\big(t_i + \gamma_4 h, w_i + \gamma_5 h f(t_i + \gamma_6 h, w_i + \gamma_7 h f(t_i, w_i))\big)\Big).$$

We need this to equal $\frac{1}{6}k_4 = \frac{h}{6} f(t_i + h, w_i + k_3)$.

So first make the time coordinate match:

$$\alpha_3 = 1.$$

Inside we need $w_i + \delta_3 h f(\ldots) = w_i + k_3$. That is,

$$\delta_3 h f(\ldots) = k_3 \implies \delta_3 = 1$$

if the inner $f(\ldots)$ equals k_3/h.

So we want

$$f\big(t_i + \gamma_4 h, w_i + \gamma_5 h f(t_i + \gamma_6 h, w_i + \gamma_7 h f(t_i, w_i))\big) = f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right) = \frac{k_3}{h}.$$

We already know that

$$k_2/h = f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)\right),$$

so we want the *innermost* f to reproduce k_2/h:

- Set

  $$\gamma_6 = \frac{1}{2}, \qquad \gamma_7 = \frac{1}{2},$$

  so that

  $$f(t_i + \gamma_6 h, w_i + \gamma_7 h f(t_i, w_i)) = f\left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)\right) = k_2/h.$$

Now the next layer's argument is

$$w_i + \gamma_5 h \cdot f(\text{inner}) = w_i + \gamma_5 k_2.$$

We want that to be $w_i + \frac{1}{2}k_2$, so

$$\gamma_5 = \frac{1}{2}.$$

And the time coordinate at this middle f should be $t_i + h/2$, so

$$\gamma_4 = \frac{1}{2}.$$

Then this middle f becomes exactly k_3/h, and with $\delta_3 = 1$ and $\alpha_3 = 1$ the outermost term is $h/6 f(t_i + h, w_i + k_3) = (1/6)k_4$.

So we get

$$\alpha_3 = 1, \quad \delta_3 = 1, \quad \gamma_4 = \frac{1}{2}, \quad \gamma_5 = \frac{1}{2}, \quad \gamma_6 = \frac{1}{2}, \quad \gamma_7 = \frac{1}{2}.$$

Final answer

Collecting everything:

$$\boxed{\begin{aligned} &\alpha_1 = \frac{1}{2}, \quad \alpha_2 = \frac{1}{2}, \quad \alpha_3 = 1, \\ &\delta_1 = \frac{1}{2}, \quad \delta_2 = \frac{1}{2}, \quad \delta_3 = 1, \\ &\gamma_2 = \frac{1}{2}, \quad \gamma_3 = \frac{1}{2}, \quad \gamma_4 = \frac{1}{2}, \quad \gamma_5 = \frac{1}{2}, \quad \gamma_6 = \frac{1}{2}, \quad \gamma_7 = \frac{1}{2}. \end{aligned}}$$

With these choices, the given nested expression is exactly the classical Runge–Kutta method of order four.

```
Dataview (inline field '==========================
Problem (a): y' = e^{t-y}, 0 <= t <= 1, y(0) = 1, h = 0.5
```

Method: Modified Euler

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	0.000000	1.000000	1.000000	0.000000e+00
1	0.500000	1.218126	1.214023	-4.103047e-03
2	1.000000	1.497555	1.489880	-7.674376e-03

Method: Midpoint

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	0.000000	1.000000	1.000000	0.000000e+00
1	0.500000	1.215430	1.214023	-1.407415e-03
2	1.000000	1.493239	1.489880	-3.358906e-03

Method: RK4

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	0.000000	1.000000	1.000000	0.000000e+00
1	0.500000	1.214041	1.214023	-1.788643e-05
2	1.000000	1.489921	1.489880	-4.116005e-05

==========

Problem (b): y' = (1+t)/(1+y), 1 <= t <= 2, y(1) = 2, h = 0.5

Method: Modified Euler

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	1.000000	2.000000	2.000000	0.000000e+00
1	1.500000	2.354167	2.354102	-6.470042e-05
2	2.000000	2.741745	2.741657	-8.769601e-05

Method: Midpoint

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	1.000000	2.000000	2.000000	0.000000e+00
1	1.500000	2.355263	2.354102	-1.161192e-03
2	2.000000	2.743513	2.741657	-1.855207e-03

Method: RK4

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	1.000000	2.000000	2.000000	0.000000e+00
1	1.500000	2.354103	2.354102	-1.261829e-06
2	2.000000	2.741659	2.741657	-1.698788e-06

==========

Problem (c): y' = -y + t*sqrt(y), 2 <= t <= 3, y(2) = 2, h = 0.25

Method: Modified Euler

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	2.000000	2.000000	2.000000	4.440892e-16
1	2.250000	2.245499	2.244121	-1.378344e-03
2	2.500000	2.567156	2.564452	-2.704017e-03
3	2.750000	2.969195	2.965194	-4.000684e-03
4	3.000000	3.456568	3.451287	-5.281770e-03

Method: Midpoint

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	2.000000	2.000000	2.000000	4.440892e-16
1	2.250000	2.244617	2.244121	-4.959881e-04
2	2.500000	2.565298	2.564452	-8.460204e-04
3	2.750000	2.966318	2.965194	-1.123920e-03
4	3.000000	3.452665	3.451287	-1.378176e-03

Method: RK4

i	t_i	w_i (approx)	y(t_i) (exact)	error y - w
0	2.000000	2.000000	2.000000	4.440892e-16
1	2.250000	2.244119	2.244121	1.695741e-06
2	2.500000	2.564449	2.564452	3.171691e-06
3	2.750000	2.965189	2.965194	4.442195e-06
4	3.000000	3.451281	3.451287	5.518090e-06

```
============================
Problem (d): y' = t^{-2}(sin(2t) - 2ty), 1 <= t <= 2, y(1) = 2, h = 0.25

Method: Modified Euler
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     1.000000            2.000000           2.000000     0.000000e+00
   1     1.250000            1.416075           1.403199    -1.287611e-02
   2     1.500000            1.031011           1.016410    -1.460092e-02
   3     1.750000            0.752267           0.738010    -1.425691e-02
   4     2.000000            0.543245           0.529687    -1.355790e-02

Method: Midpoint
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     1.000000            2.000000           2.000000     0.000000e+00
   1     1.250000            1.436510           1.403199    -3.331131e-02
   2     1.500000            1.051674           1.016410    -3.526370e-02
   3     1.750000            0.769358           0.738010    -3.134848e-02
   4     2.000000            0.556679           0.529687    -2.699153e-02

Method: RK4
   i        t_i           w_i (approx)      y(t_i) (exact)      error y - w
-------------------------------------------------------------------------------
   0     1.000000            2.000000           2.000000     0.000000e+00
   1     1.250000            1.403357           1.403199    -1.576461e-04
   2     1.500000            1.016559           1.016410    -1.484392e-04
   3     1.750000            0.738132           0.738010    -1.219121e-04
   4     2.000000            0.529786           0.529687    -9.846674e-05'): Error:
-- PARSING FAILED -----------------------------------------------

> 1 | ============================
    | ^
  2 | Problem (a): y' = e^{t-y}, 0 <= t <= 1, y(0) = 1, h = 0.5
  3 |

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```